

Accelerating BERT-base training on a single A100

HPC Tools, Lab AI: deliverable 1 (BASELINE)

Nicolás Aller Ponte

30 September 2026

Abstract

We fine-tune BERT-base for question answering on SQuAD v1.1 on one NVIDIA A100 of FinisTerra III and apply the single-GPU optimizations of the course one at a time, profiling each of them. Two epochs take 45.8 minutes in the FP32 baseline and 7.2 minutes once optimized, a $6.4\times$ speedup, while F1 drops by one point (87.97 to 86.97). Precision is the decisive lever; the remaining optimizations each remove the bottleneck exposed by the previous one.

1 Setup and method

The model is `google-bert/bert-base-uncased` (109 M parameters), trained for 2 epochs with AdamW (learning rate $3 \cdot 10^{-5}$, 10 % linear warmup, then linear decay) on 88,492 training windows of 384 tokens. All runs use one A100-PCIE-40GB and 32 CPU cores, with PyTorch 2.11 built for CUDA 12.8.

- **Throughput** is measured over 100 steps after a warmup of 30 steps (60 with `torch.compile`), between two `torch.cuda.synchronize()` calls.
- Each configuration runs **three times**; we report the median and the range.
- `torch.profiler` records five steps per configuration. We report the *share* of GPU time per kernel group: shares are comparable across configurations, whereas absolute times are not, since the profiled steps are the first ones of each run.
- The baseline is true FP32 (TF32 disabled), and quality is measured with the official SQuAD exact match and F1 on the full validation set.

2 Deployment on FinisTerra III

Most of the effort went into running the code correctly on the cluster (Table 1).

Table 1: Deployment issues and their solutions.

Issue	Solution
The default PyTorch wheel targets CUDA 13; the A100 driver (570.86.15) supports up to 12.8	Install PyTorch from the CUDA 12.8 index
<code>python/3.10.10</code> is not available	<code>module load cesga/2020 python/3.10.8</code>
A GPU job with 4 cores is rejected	Exactly 32 cores per A100
<code>\$HOME</code> quota is 20 GB	Virtual environment and Hugging Face cache in <code>\$STORE</code>
Tokenizing in the training job would count as training time	A separate CPU job tokenizes once
Long jobs queue longer	Jobs under 6 h run in the high-priority short QoS

3 Results

3.1 One optimization at a time

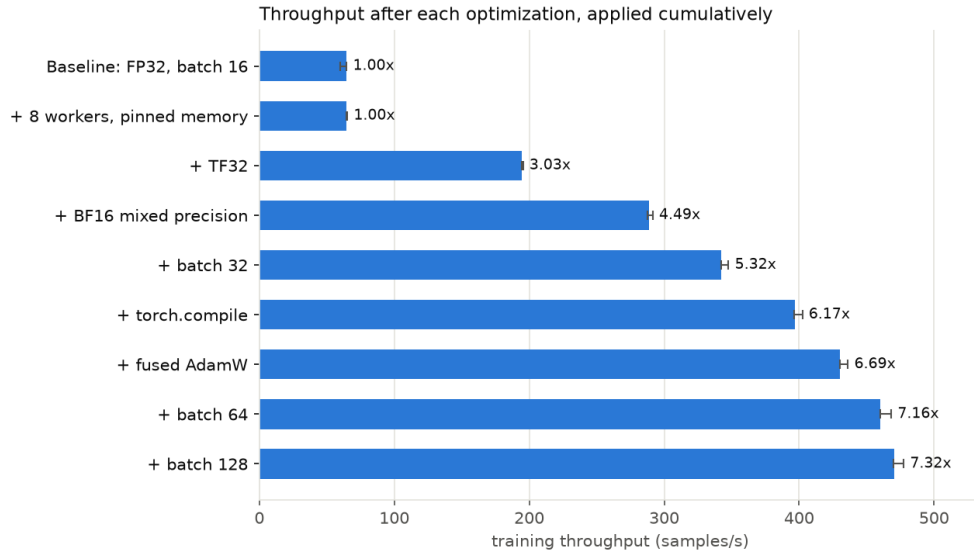


Figure 1: Training throughput after each cumulative optimization. Bars show the median of three runs and whiskers the range; labels give the speedup over the baseline.

Table 2: Throughput and peak memory of each step (median and range of three runs).

Step	Change	Samples/s [range]	Speedup	Memory
0	Baseline: FP32, batch 16, no workers	64.3 [60.0, 64.7]	1.00×	5.3 GB
1	+ 8 workers, pinned memory	64.4 [64.4, 65.2]	1.00×	5.3 GB
2	+ TF32	194.6 [194.6, 195.4]	3.03×	5.3 GB
3	+ BF16 mixed precision	289.0 [287.5, 291.5]	4.49×	4.1 GB
4	+ batch 32	342.2 [342.2, 347.7]	5.32×	6.6 GB
5	+ <code>torch.compile</code>	397.0 [396.1, 402.4]	6.17×	6.0 GB
6	+ fused AdamW	430.1 [430.0, 436.2]	6.69×	6.0 GB
7	+ batch 64	460.2 [460.0, 468.3]	7.16×	10.4 GB
8	+ batch 128	470.4 [470.2, 477.5]	7.32×	19.3 GB

3.2 Where the GPU time goes

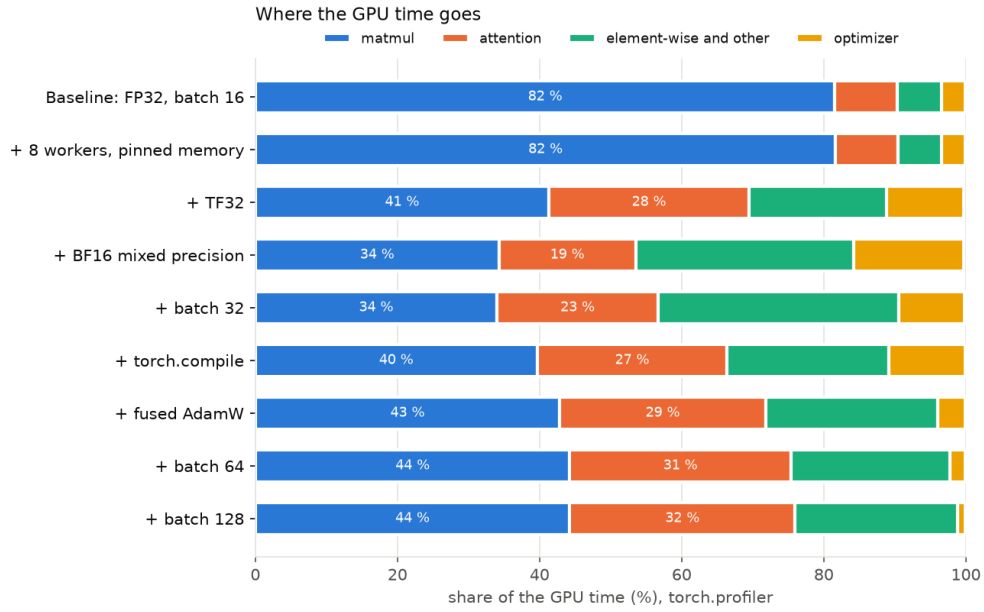


Figure 2: Share of GPU time per kernel group, from the profiler traces. Kernels launched inside the optimizer step are counted as optimizer. Memory copies stay below 0.5 %.

3.3 Full training

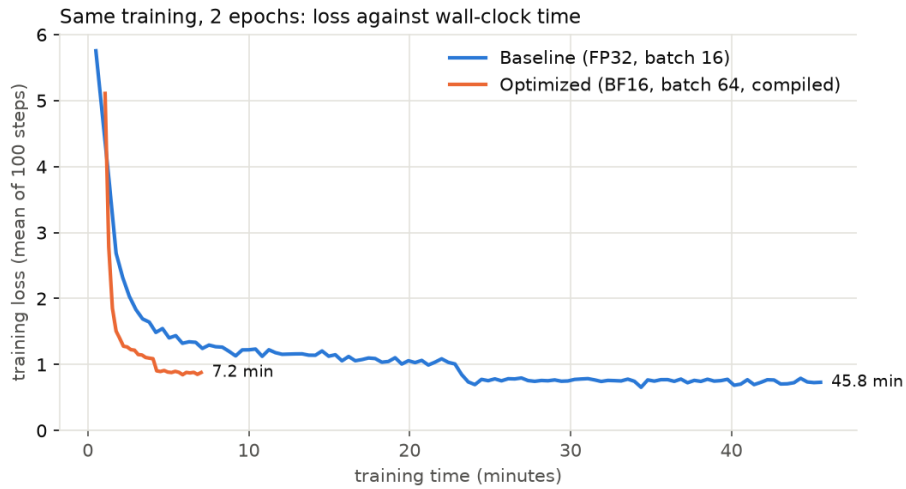


Figure 3: Training loss against wall-clock time for the two full trainings. The optimized run starts about a minute later because of compilation.

Table 3: Full training, 2 epochs.

	Baseline (step 0)	Optimized (step 7)
Training time	2,747 s (45.8 min)	431 s (7.2 min), 6.37×
Mean throughput	64.4 samples/s	410.1 samples/s
Exact match / F1	80.34 / 87.97	79.13 / 86.97
Peak memory	5.3 GB	10.4 GB

4 Conclusions

- **The input pipeline was never the bottleneck.** The data is tokenized in advance and held in memory, so extra workers and pinned memory have nothing to hide (1.00 $\times$).
- **Precision is the decisive lever.** In FP32, 82 % of the GPU time goes to matrix products on the CUDA cores. TF32 moves them to the Tensor Cores without touching the model (3.0 $\times$), and BF16 also halves the data moved (4.5 $\times$). BF16 keeps the FP32 exponent range, so no loss scaling is needed.
- **Each optimization exposes the next bottleneck.** Once matrix products are fast, element-wise kernels and the optimizer weigh more: `torch.compile` fuses the former (+16 %), and fused AdamW cuts the optimizer from 11 % to 4 % of the GPU time (+8 %). Larger batches spread fixed per-step costs with diminishing returns: batch 128 gains only 2 % over 64 for twice the memory, so the final run uses 64.
- **Attention is now the main cost (32 %).** PyTorch selects the memory-efficient attention kernel rather than FlashAttention, because BERT passes a padding mask; its backward pass alone takes 23 % of the GPU time. This is where the next gain lies.
- **Time improves less than throughput** (6.4 $\times$ against 7.3 $\times$). The optimized run spends about 56 s outside the steady state, mostly compiling, a fixed cost that weighs more in a 7-minute run than in a longer one.
- **Quality is preserved.** The baseline matches the published BERT-base figures (80.8 EM, 88.5 F1). The optimized run loses about one point, plausibly because batch 64 makes four times fewer optimizer updates at the same learning rate; tuning it is outside the scope of this deliverable.
- **Utilization.** The baseline reaches 67 % of the FP32 peak and the optimized run 31 % of the BF16 peak, which is 16 times higher: 96 against 13 TFLOP/s, counting 204 GFLOP per training sample (Kaplan et al., 2020).